

Project Report: Multi-Agent System Supporting Software Development Processes

1. Introduction

This report presents an analysis and the results of a research and development project focused on creating a **multi-agent system capable of autonomously generating code based on a given project scope**. The system was designed using a **ReAct agent architecture** and implemented in **Python** using the **LangGraph** library for agent orchestration and **Streamlit** for the graphical user interface (GUI). A key element of the system was the application of various reasoning strategies within the code generation agent (Coder) to evaluate their impact on the quality and efficiency of the generated software.

2. System Architecture

The system comprises four main types of agents, collaborating to achieve project goals:

- **Project Manager:** Receives the project scope from the user, plans tasks, divides them into sprints, and assigns them to the appropriate agents. It coordinates the workflow, iterating through tasks and ensuring their chronological execution.
- **Researcher:** Responsible for gathering necessary information, exploring technologies, and identifying solutions required for task completion.
- **Coder:** The primary agent responsible for code generation. It features three reasoning versions, allowing for experimentation with different approaches to breaking down larger tasks into smaller subtasks and executing them sequentially:
 - **ReAct (No Reasoning):** A basic approach where the agent operates on observation, thought, and action.
 - **ReAct + CoT (Chain-of-Thought Reasoning):** The agent generates internal "thoughts" that help it in reasoning and planning subsequent steps.
 - **ReAct + ToT (Tree-of-Thought Reasoning):** The agent explores multiple reasoning paths, evaluating them and selecting the best one.
- **Documenter:** Creates documentation for the generated code, including function descriptions, usage instructions, and other essential materials.

2.1. Operational Flow

The system's operational flow is as follows:

1. **User defines project scope:** The user provides a general description of the project to be completed.
2. **Project Manager plans:** The Project Manager analyzes the scope, breaks it down into smaller, manageable tasks, and organizes them into sprints. Each task is assigned to the appropriate agent.
3. **Task iteration and execution:** The Project Manager iterates through the planned tasks. Depending on the task type, the relevant agent (Researcher, Coder, Documenter) is activated and executes its task chronologically.
4. **Code generation and verification:** The Coder generates code according to requirements. Depending on the chosen Coder version, different reasoning strategies are employed to optimize the process.

5. **Documentation:** The Documenter creates comprehensive documentation for the generated code.

3. Test Projects

Six defined projects, categorized into three types, were used to test the system, allowing for a comprehensive evaluation of its capabilities:

3.1. Simple Console Applications

- **Project 1: Temperature Converter (Single-file):** A simple temperature unit converter (°C, °F, K) with a CLI interface and error handling.
- **Project 2: CLI Note Manager (Multi-file):** A command-line note manager allowing note creation, deletion, and viewing. It requires a modular structure with multiple files (`main.py`, `note.py`, `storage.py`, `utils.py`).

3.2. Games

- **Project 3: Tic Tac Toe:** A classic 3x3 game for the terminal (2 players or player vs. AI) with ASCII board rendering and win/draw detection.
- **Project 4: Snake Game with GUI:** A classic Snake game with a Pygame GUI, keyboard controls, collision detection, and visual board/score display.

3.3. Web Applications

- **Project 5: TODO List (Backend + simple frontend):** A web application for task management with a REST API (Flask, Django, or FastAPI) and a simple frontend (HTML+JS).
 - **Project 6: Weather App (frontend querying external API):** A web application displaying weather data for a selected city, fetching data from a public API (e.g., OpenWeatherMap) and dynamically rendering the information.
-

4. Measurement Metrics

To assess the performance and quality of the generated code, the following metrics were used:

- **Time of generation (s):** The time taken for the agent to generate the code.
 - **Number of lines of code:** The total count of code lines.
 - **Number of used tokens:** The count of tokens consumed by the language model.
 - **Number of lines of code after changes:** The count of code lines after any manual corrections.
 - **Levenshtein distance:** The Levenshtein distance from the "ideal" solution (closer to 0 is better).
 - **Cyclomatic complexity (avg):** The average cyclomatic complexity of the code, measuring control flow complexity.
 - **Code duplication (%):** The percentage of duplicated code.
 - **Lint errors (Python):** The number of errors reported by a Python linter (PEP 8).
 - **Notes:** Additional remarks concerning issues or specific characteristics of the generated code.
-

5. Results and Analysis

The results for the three Coder versions—**No Reasoning (ReAct)**, **CoT Reasoning (ReAct + CoT)**, and **ToT Reasoning (ReAct + ToT)**—are presented below. All tests were conducted using the **gpt-4.1** model.

5.1. v0.2 - CoT Reasoning

Metric	Project 1	Project 2	Project 3	Project 4	Project 5	Project 6
Time of generation (s)	171.98	612.75	459.72	625.31	860.55	385.27
Number of lines of code	128	377	400	292	657	252
Number of used tokens	102095	314406	237500	324657	445496	207828
Number of lines of code after changes	128	377	400	292	678	252
Levenshtein distance	0	0	0	0	293	146
Cyclomatic complexity (avg)	2.2	2.6	2.9	2.2	1.4	2
Code duplication (%)	0	0	0	0	0	0
Lint errors (Python)	0	4	1	0	1	N/A
Notes					Added scripts to html, removed exports	Added API key, removed exports

CoT Reasoning Analysis: The model with CoT reasoning demonstrated **relatively long generation times**, especially for more complex projects (P2, P4, P5). Nevertheless, for simple tasks (P1, P2, P3, P4), the code was **functionally correct** (Levenshtein distance = 0). For P5 and P6, the system generated code requiring **manual corrections**, primarily related to frontend configuration (adding scripts to HTML, removing exports) and API keys. Low average cyclomatic complexity indicates **well-structured code**, and the number of linting errors was minimal.

5.2. v1.1 - No Reasoning

Metric	Project 1	Project 2	Project 3	Project 4	Project 5	Project 6
Time of generation (s)	92.25	167.72	450.09	200.06	287.23	196.37
Number of lines of code	77	188	372	227	313	243
Number of used tokens	45590	85116	234158	95994	171589	112006

Metric	Project 1	Project 2	Project 3	Project 4	Project 5	Project 6
Number of lines of code after changes	N/A	191	N/A	227	322	238
Levenshtein distance	0	61	0	31	140	142
Cyclomatic complexity (avg)	5.25	3.2	3.65	2.44	1.81	N/A
Code duplication (%)	0%	0%	0%	0%	0%	0%
Lint errors (Python)	1	0	0	0	0	N/A
Notes		Function to validate input was badly used	Agent added ability to select O or X for Human vs AI game	Created dir 'snake_game' which is useless	CORS Errors	

No Reasoning Analysis: The version without additional reasoning was **significantly faster** in code generation and consumed **fewer tokens**. However, this resulted in **lower code correctness** for more complex projects (P2, P4, P5, P6), reflected by a higher Levenshtein distance. For example, in P2, the input validation function was improperly used, and P5 encountered **CORS errors**. In some cases (P4), **unnecessary directories were created**. Cyclomatic complexity varied, and the number of linting errors was generally low. This approach proved most effective for **very simple and precisely defined tasks**.

5.3. v1.2 - ToT Reasoning (Updated)

Metric	Project 1	Project 2	Project 3	Project 4	Project 5	Project 6
Time of generation (s)	300.82	562.42	743.00	707.95	699.62	552.51
Number of lines of code	91	255	809	801	690	443
Number of used tokens	208949	441787	711946	661526	545036	393723
Number of lines of code after changes	N/A	255	N/A	N/A	690	436

Metric	Project 1	Project 2	Project 3	Project 4	Project 5	Project 6
Levenshtein distance	0	4	0	0	23	575
Cyclomatic complexity (avg)	3.8	2.23	3.07	2.34	1.5	N/A
Code duplication (%)	0%	0%	0%	0%	0%	0%
Lint errors (Python)	1	5	5	0	1	N/A
Notes	No ability to remove tasks on frontend. Needed to fix CORS (set to all hosts)					

ToT Reasoning Analysis: The ToT Reasoning strategy continued to show the **longest generation times** and **highest token consumption** among the methods, aligning with its iterative and tree-like reasoning. While it performed well for Projects 1, 3, and 4 with a Levenshtein distance of 0, indicating **perfect functional correctness**, Projects 2 and 5 still had minor issues, such as the lack of task deletion functionality on the frontend in P5 and the necessity for manual CORS configuration. Notably, Project 6 showed a **very high Levenshtein distance (575)**, suggesting significant functional discrepancies or incompleteness, which highlights a key area for improvement. The larger number of code lines in Projects 3, 4, and 5 might imply more verbose or comprehensive solutions. Lint errors were present but manageable.

6. Comparison of Reasoning Strategies

Metric	No Reasoning (ReAct)	CoT Reasoning (ReAct + CoT)	ToT Reasoning (ReAct + ToT)
Generation Time	Fastest	Medium	Slowest
Code Correctness	Lowest (often requires fixes)	Medium (good for simple, needs fixes for complex)	Highest (good for some complex, but inconsistent for others)
Code Complexity	Varied	Low/Medium	Low/Medium
Token Usage	Lowest	Medium	Highest
Stability/Consistency	Lowest	Medium	Mixed (High for some, low for others)

Conclusions from Comparison:

- No Reasoning (ReAct):** This approach is the **fastest** and consumes the **fewest tokens**. However, it tends to generate **lower-quality code** that often requires manual intervention, especially for complex

projects. It's best suited for very simple, well-defined tasks where speed is paramount.

- **CoT Reasoning (ReAct + CoT):** Offers **better code quality** than "No Reasoning," striking a good balance between generation time and token consumption. It's a solid choice for **moderately complex projects**.
 - **ToT Reasoning (ReAct + ToT):** Despite having the **longest generation times** and **highest token consumption**, ToT Reasoning often generates **highly correct solutions for complex problems** (e.g., Project 3 and 4). However, the results for Project 6 show that its performance can be **inconsistent across different complex projects**, sometimes leading to significant functional errors. This highlights that while ToT has the potential for robust solutions through its iterative reasoning, it might still struggle with specific task complexities or nuances, indicating areas for further refinement.
-

7. Final Conclusions

The project to create a multi-agent autonomous code generation system has demonstrated that a multi-agent approach with ReAct agents is **promising**. The different reasoning strategies within the Coder agent have a **significant impact on the efficiency and quality of the generated code**. The choice of the appropriate strategy should depend on the **complexity of the project and the priorities** (e.g., generation speed vs. code quality).

Future work could involve further optimization of reasoning strategies, the introduction of automated code verification and testing mechanisms, and the expansion of other agents' roles (e.g., a debugging agent). Developing these areas could significantly contribute to increasing the autonomy and effectiveness of such systems.